NutriLens AI

An Evidence-Based Food Analysis and Nutritional Scoring Agent

EECS 3311 — Software Design
Course Project: Stage 1 Design Report

Student: Jacob Silva
Student Number: 220594347
Instructor: Prof. Song Wang
Submission Date: 10/09/2026

Contents

1. Project Overview
2. Detailed Feature Specifications
3. UML Class Diagram
4. Design Pattern Explanations
5. UML Use-Case Diagram
6. Detailed Use-Case Descriptions
7. Sequence Diagrams
8. Feature-to-Design Traceability
9. Feature Implementation Explanations

Appendix A. Proposed Class Inventory

Appendix B. Finalized Scoring Model and Planned Technology Choices

1. Project Overview

1.1 Problem and Motivation

Consumers are frequently presented with packaged foods containing complex nutrition labels and long ingredient lists. Although nutritional information is technically available, interpreting the relationship between macronutrients, micronutrients, serving sizes, ingredients, and scientific research can require significant background knowledge. Consumers may also encounter conflicting or oversimplified claims when searching for information online.

NutriLens AI is designed to make food-label information easier to understand. A user can upload or photograph a packaged food label, or enter the information manually. The system structures the nutrition data and ingredient list, analyzes the product, calculates a transparent 1–100 nutritional score, retrieves relevant evidence for ingredients, and explains the result in clear language. The user can then ask follow-up questions, compare products, calculate macros for custom serving amounts, and receive automatic alternative suggestions.

1.2 Target Users

The primary users are consumers who want to better understand packaged foods without manually interpreting every nutrient value or researching every ingredient. The application is intended for informational and educational use. It is not intended to diagnose medical conditions, prescribe treatment, or replace professional medical advice.

1.3 Agent Description and AI Integration

NutriLens AI is an AI-agent-based software system rather than a simple chatbot. The AI component participates in retrieval, reasoning, evidence summarization, tool selection, contextual follow-up, and recommendation generation. Deterministic components remain responsible for tasks that should be reproducible, including input validation, serving-size calculations, and the numerical scoring algorithm.

- Planned AI/LLM provider: OpenAI API accessed through the LLMClient adapter.
- AI responsibilities: interpret user questions, plan research actions, summarize retrieved evidence, explain scores, recommend alternatives, and maintain conversational context.
- Deterministic responsibilities: validate data, parse structured values, calculate serving-adjusted macros, and calculate the 1–100 score using defined rules.
- Planned external tools/services: Tesseract OCR for text extraction, PubMed / Europe PMC for research retrieval, Open Food Facts for product data, and adapter-based integrations for all external providers.

NutriLens AI uses a deterministic 1–100 scoring model. The score begins from a neutral base score of 50 and is adjusted using positive nutrition factors, negative nutrition factors, and ingredient-quality penalties. Positive factors include fibre, protein, and selected micronutrients, while negative factors include added sugar, sodium, and saturated fat. Ingredient quality adjustments penalize highly processed ingredients, artificial additives or low-quality fillers, and excessive ingredient complexity. Missing values are marked unavailable rather than invented, and scientific evidence primarily supports explanation and ingredient discussion rather than directly generating the numeric score.

The planned technology choices for Stage 2 implementation are JavaFX for the GUI, a Java console menu interface for the CLI, OpenAI API for LLM-based reasoning and explanation, Tesseract OCR for label-text extraction, PubMed / Europe PMC for research retrieval, Open Food Facts for product and alternative-search data, and SQLite or a JSON-based repository for persistence. All external integrations are accessed through adapters so providers can be replaced without changing core business logic.

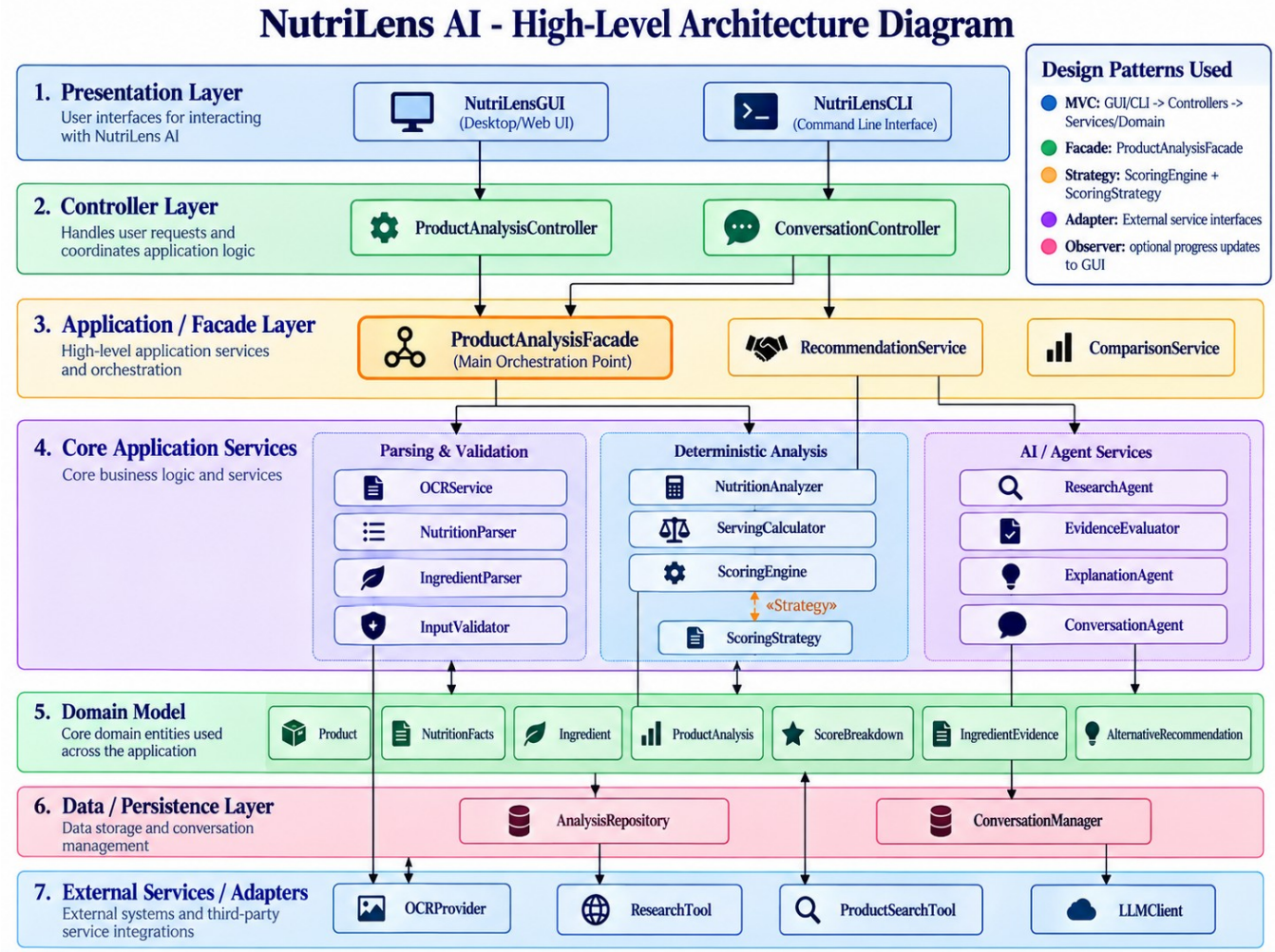
1.4 GUI and CLI

The project will provide both a graphical user interface (GUI) and a command-line interface (CLI). Both interfaces will access the same controller and service layer so that application logic is not duplicated.

- GUI: product image upload, manual correction, score dashboard, nutrition/ingredient tabs, comparison view, alternatives, history, and conversational question panel.
- CLI: menu-based access to product analysis, manual entry, ingredient research, comparison, alternatives, macro calculations, follow-up questions, and history.

1.5 Overall Architecture

The proposed architecture separates presentation, coordination, deterministic analysis, AI-agent behavior, data storage, and external integrations. GUI and CLI views communicate with controllers. A ProductAnalysisFacade coordinates OCR, parsing, research, scoring, and explanation. External services are hidden behind interfaces/adapters so they can be changed without rewriting the core application.



2. Detailed Feature Specifications

The following features are intentionally distinct and non-trivial. Together they cover image/manual input, deterministic analysis, AI-agent behavior, retrieval, memory, recommendation, comparison, persistence, and calculation.

F01 — Scan / Upload Food Label

Description	Allows the user to photograph or upload an image containing a Nutrition Facts panel and/or ingredient list.
User Interaction	GUI: drag-and-drop, file picker, or camera action. CLI: provide an image file path.
Input	Image file containing a food label.
Output	Image-processing request plus extracted raw text and confidence information for downstream parsing.
AI Involvement	Hybrid. OCR/tool processing is used to extract text; AI may assist with resolving ambiguous label structure, but extracted values must remain reviewable.
Expected Workflow	User selects an image → image is validated → OCR service extracts text → parser attempts to identify nutrition and ingredient sections → extracted data is shown for confirmation/correction.
Error / Alternative Cases	Unsupported file type, unreadable image, missing label region, low OCR confidence, or service failure. The system should request a clearer image or offer manual entry.

F02 — Manual Product Entry & OCR Correction

Description	Allows users to manually enter product information or correct values that were extracted incorrectly from a label image.
User Interaction	Editable form in the GUI; prompted fields in the CLI.
Input	Product name, serving size, nutrient values, ingredient list, and any corrected OCR values.
Output	Validated structured product data.
AI Involvement	Deterministic. Validation and field updates should follow explicit rules rather than LLM judgment.
Expected Workflow	User enters or edits values → InputValidator checks format/range → errors are highlighted → valid data is converted into domain objects.
Error / Alternative Cases	Missing required fields, negative quantities, invalid units, impossible formats, or incomplete serving information. The system should identify the exact field requiring correction.

F03 — Nutrition Profile Analysis

Description	Analyzes the product's available nutrition information, including calories, macronutrients, fibre, sugars, sodium, and declared micronutrients.
User Interaction	Displayed in a nutrition-analysis panel after product data is confirmed.
Input	Validated NutritionFacts data and serving information.
Output	Structured nutrition analysis identifying notable nutrient characteristics and normalized values.
AI Involvement	Primarily deterministic. AI may later explain the results, but numeric analysis is calculated from structured data.
Expected Workflow	NutritionFacts are normalized → nutrient values are evaluated using defined rules → analysis metrics are stored in ProductAnalysis → results are displayed.
Error / Alternative Cases	Missing nutrients, unsupported units, or incomplete label data. Missing values should be marked unavailable rather than inferred.

F04 — Ingredient Analysis

Description	Parses the ingredient list, separates individual ingredients, normalizes common naming variations, and provides a structured ingredient profile.
User Interaction	Ingredient tab/list showing recognized ingredients and selectable entries.
Input	Raw ingredient-list text.
Output	Ordered Ingredient objects, normalized names, and analysis flags where applicable.
AI Involvement	Hybrid. Deterministic parsing handles common separators and formats; AI may help interpret unusual or ambiguous ingredient names.
Expected Workflow	Ingredient text is parsed → names are normalized → ambiguous entries are flagged → ingredients become available for research and question answering.
Error / Alternative Cases	Malformed lists, nested sub-ingredients, unknown names, or ambiguous OCR. The system should preserve the original text and flag uncertain parsing.

F05 — Scientific Evidence Retrieval

Description	Retrieves research or authoritative information relevant to ingredients selected by the system or user.
User Interaction	Automatically triggered during analysis when needed; also accessible through an ingredient 'Research' action.
Input	Normalized ingredient name, user question, and research intent.
Output	ResearchResult objects containing retrieved sources, metadata, and relevant text snippets.
AI Involvement	AI/Hybrid. The agent decides when retrieval is needed, generates search intent/queries, and invokes the configured research tool.
Expected Workflow	ResearchAgent receives ingredient/question → plans a search → ResearchTool retrieves information → results are returned to the agent for evaluation.
Error / Alternative Cases	No results, irrelevant results, unavailable service, malformed response, or insufficient evidence. The system should report that evidence is unavailable rather than fabricate it.

F06 — Evidence Evaluation & Summary

Description	Evaluates retrieved information for relevance and summarizes the evidence, including uncertainty or disagreement between sources.
User Interaction	Evidence section with concise summary and supporting source entries.
Input	Retrieved ResearchResult collection.
Output	IngredientEvidence containing a summary, relevance notes, confidence/uncertainty, and source references.
AI Involvement	AI-based with deterministic safeguards. The LLM summarizes and compares evidence; source presence and required fields are validated by software.
Expected Workflow	Retrieved results are filtered/validated → EvidenceEvaluator compares relevant findings → agent generates a grounded summary → source references are displayed.
Error / Alternative Cases	Conflicting sources, weak evidence, irrelevant retrieval, or missing citations. The system should communicate uncertainty and avoid unsupported conclusions.

F07 — 1–100 Nutritional Score

Description	Calculates a reproducible nutritional-quality score from 1 to 100 using explicitly defined scoring rules.
User Interaction	Prominent score card shown after analysis.
Input	Validated nutrition analysis, ingredient characteristics, and finalized scoring parameters.
Output	Numeric score plus component-level contributions used by the explanation feature.
AI Involvement	Deterministic. The LLM does not choose the numeric score.
Expected Workflow	ScoringEngine receives analysis data → active ScoringStrategy applies weights/rules → contributions are combined → score is bounded to 1–100 → breakdown is stored.
Error / Alternative Cases	Insufficient required data or invalid scoring inputs. The system should report that a complete score cannot be produced or use a documented reduced-data rule.

F08 — Explainable Score Breakdown

Description	Explains which measurable factors raised or lowered the product's score and translates the deterministic breakdown into understandable language.
User Interaction	Score details panel and 'Why this score?' action.
Input	ScoreBreakdown, nutrition analysis, ingredient analysis, and available evidence.
Output	Human-readable explanation linked to actual scoring factors.
AI Involvement	Hybrid. Deterministic score contributions are the source of truth; AI generates the natural-language explanation.
Expected Workflow	ScoreBreakdown is retrieved → explanation prompt is built from verified factors → LLMClient generates explanation → response is checked for consistency with the breakdown.
Error / Alternative Cases	LLM response contradicts scoring data, explanation service fails, or score breakdown is unavailable. The system should fall back to a deterministic factor list.

F09 — Product Comparison

Description	Compares two previously analyzed products across score, nutrients, ingredients, and relevant evidence.
User Interaction	Comparison screen/table; CLI prompts the user to select two products.
Input	Two ProductAnalysis records.
Output	Side-by-side comparison and summarized differences/trade-offs.
AI Involvement	Hybrid. Numeric comparisons are deterministic; AI may explain trade-offs in natural language.
Expected Workflow	User selects two products → comparison metrics are calculated → scores/nutrients are displayed side by side → optional AI summary explains major trade-offs.
Error / Alternative Cases	Products use incompatible serving bases, missing data, or one product is not analyzed. The system should normalize when possible and clearly mark unavailable comparisons.

F10 — Ingredient Question Answering

Description	Lets the user ask a targeted natural-language question about an ingredient, such as its function or what available research says.
User Interaction	Question box associated with an ingredient or general chat panel.
Input	Ingredient context plus natural-language question.
Output	Grounded answer using existing evidence and/or newly retrieved research.
AI Involvement	AI-based. The agent interprets the question, decides whether retrieval is necessary, and generates the answer.
Expected Workflow	Question is interpreted → current ingredient context is loaded → retrieval is invoked if needed → evidence is summarized → answer is returned.
Error / Alternative Cases	Ambiguous ingredient reference, insufficient evidence, failed retrieval, or question outside project scope. The agent should request clarification or state limitations.

F11 — Contextual Follow-Up Conversation

Description	Maintains the current product and analysis context so the user can ask follow-up questions without repeating information.
User Interaction	Persistent conversation panel in the GUI; conversational prompt mode in the CLI.
Input	User message plus conversation/product context.
Output	Context-aware response and updated conversation history.
AI Involvement	AI-based with memory/context management.
Expected Workflow	ConversationManager retrieves current context → AgentController interprets the question → appropriate analysis/research tools are used → response is generated → conversation state is updated.
Error / Alternative Cases	Missing context, product changed unexpectedly, context too large, or ambiguous pronouns/references. The agent should ask the user to clarify which product or ingredient is intended.

F12 — Analysis History / Saved Products

Description	Stores completed analyses so users can reopen products, reuse them in comparisons, and continue later.
User Interaction	History/saved-products panel in the GUI; numbered saved-product list in the CLI.
Input	Completed ProductAnalysis records and optional user-provided product name.
Output	Persisted records that can be listed, loaded, and removed.
AI Involvement	Deterministic.
Expected Workflow	Completed analysis is saved → repository assigns an identifier → history view lists records → user can reopen or select records for comparison.
Error / Alternative Cases	Persistence failure, corrupted record, duplicate names, or missing saved item. The system should retain the current unsaved analysis and report the storage problem.

F13 — Automatic Alternative Recommendations

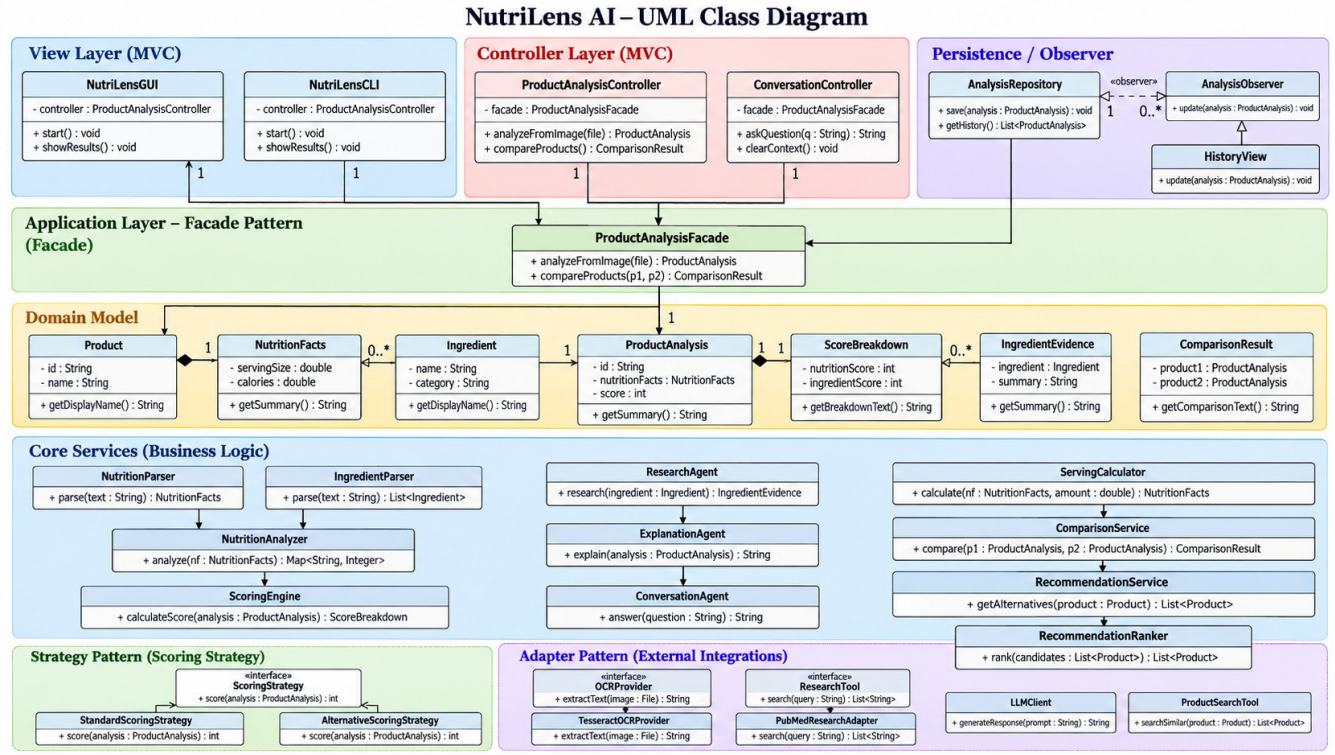
Description	Automatically recommends comparable products with stronger nutritional characteristics after an analysis and explains why they may be better alternatives.
User Interaction	Alternatives panel shown after analysis with ranked suggestions; CLI option to view recommended alternatives.
Input	Current ProductAnalysis, product category/characteristics, candidate product data, and user constraints if available.
Output	Ranked AlternativeRecommendation list with score/nutrition differences and explanation.
AI Involvement	Hybrid. Retrieval finds real candidate products, deterministic scoring evaluates candidates, and AI explains the trade-offs/recommendation.
Expected Workflow	Agent identifies comparable-product criteria → ProductSearchTool retrieves candidates → candidates are analyzed/scored → RecommendationRanker orders them → ExplanationAgent explains the top alternatives.
Error / Alternative Cases	No comparable products, incomplete candidate data, retrieval failure, or alternatives that violate user-stated constraints. The system should explain why no confident recommendation is available.

F14 — Macro Calculator & Serving Adjustment

Description	Calculates calories and macronutrients for a custom serving size or number of servings rather than only displaying the label's default serving.
User Interaction	Custom serving control beside the nutrition profile; CLI prompts for grams or number of servings.
Input	Label serving size, nutrient values per serving, and requested serving amount.
Output	Adjusted calories, protein, carbohydrates, fat, fibre, sugars, and other scalable values.
AI Involvement	Deterministic.
Expected Workflow	User enters amount → ServingCalculator converts it to a serving multiplier → nutrient values are scaled → rounded display values are returned.
Error / Alternative Cases	Zero/negative serving amount, incompatible units, missing label serving size, or non-scalable values. The system should reject invalid input and explain what information is required.

3. UML Class Diagram

The class diagram includes the major presentation, controller, domain, service, AI-agent, persistence, and external-tool interfaces used by NutriLens AI. It shows key attributes and methods, multiplicities, associations, dependencies, inheritance, aggregation/composition, and the selected design patterns.



3.1 Proposed Core Class Groups

Layer / Group	Proposed Classes / Interfaces
Presentation	NutriLensGUI, NutriLensCLI
Controllers	ProductAnalysisController, ConversationController
Facade / Application Services	ProductAnalysisFacade, ComparisonService, RecommendationService
Domain	Product, NutritionFacts, Ingredient, ProductAnalysis, ScoreBreakdown, IngredientEvidence, AlternativeRecommendation
Parsing / Validation	OCRService, NutritionParser, IngredientParser, InputValidator
Deterministic Analysis	NutritionAnalyzer, ScoringEngine, ServingCalculator, RecommendationRanker
AI / Agent	ResearchAgent, ExplanationAgent, ConversationAgent, EvidenceEvaluator
External Interfaces	LLMClient, ResearchTool, ProductSearchTool, OCRProvider
Persistence	AnalysisRepository, ConversationManager

4. Design Pattern Explanations

The design uses five patterns because each solves a specific architectural problem. Each pattern is represented in the class diagram and mapped to the classes that participate in it.

4.1 MVC (Model–View–Controller)

Design problem addressed	The project must support both GUI and CLI views without duplicating business logic.
Participating classes	Views: NutriLensGUI, NutriLensCLI. Controllers: ProductAnalysisController, ConversationController. Models/domain: Product, NutritionFacts, ProductAnalysis, IngredientEvidence.
Roles	Views collect/display information, controllers coordinate requests, and model/service classes contain application state and logic.
Why appropriate	MVC cleanly separates presentation from core behavior and allows GUI and CLI to reuse the same services.
What becomes harder without it	Without MVC, interface code would become tightly coupled to scoring, research, and persistence logic, making two user interfaces difficult to maintain.

4.2 Strategy

Design problem addressed	The scoring design may evolve, and different scoring policies should be replaceable without changing the ScoringEngine client code.
Participating classes	ScoringStrategy interface, StandardScoringStrategy (and future alternatives), ScoringEngine.
Roles	ScoringEngine delegates the scoring algorithm to the active strategy.
Why appropriate	Strategy supports experimentation and keeps scoring rules modular and testable.
What becomes harder without it	Without Strategy, scoring logic would be embedded in a large conditional method, making future changes risky and hard to test.

4.3 Adapter

Design problem addressed	OCR, research, product-search, and LLM providers expose different APIs and data formats.
Participating classes	Interfaces such as OCRProvider, ResearchTool, ProductSearchTool, LLMClient plus provider-specific adapter implementations.
Roles	Adapters translate provider-specific requests/responses into NutriLens domain-friendly structures.
Why appropriate	Adapter isolates external dependencies and makes service replacement possible.

What becomes harder without it	Without Adapter, provider-specific JSON, authentication, and response formats would leak throughout the application.

4.4 Facade

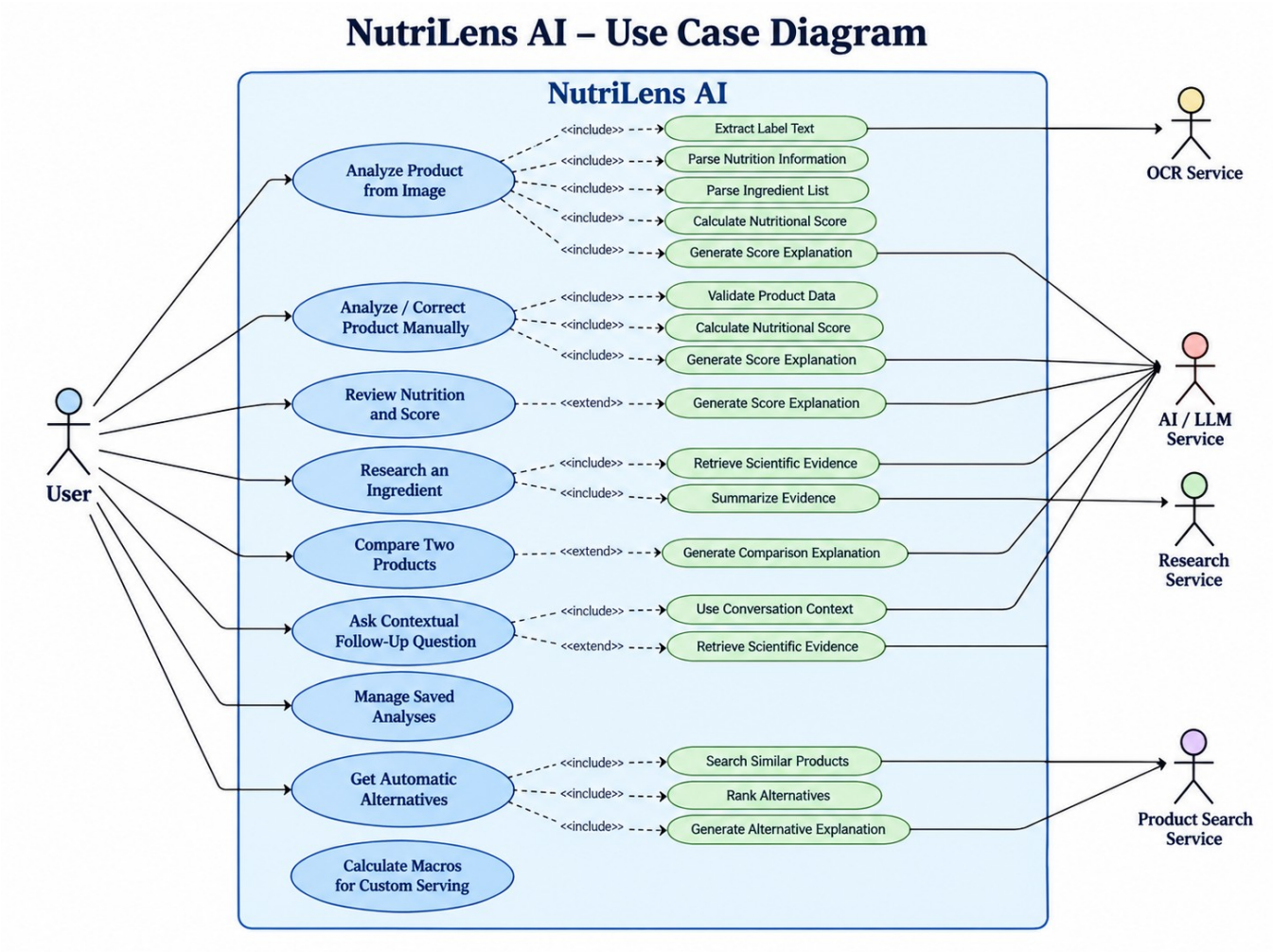
Design problem addressed	A complete product analysis requires many subsystems: OCR, parsing, validation, nutrition analysis, research, scoring, and explanation.
Participating classes	ProductAnalysisFacade plus the underlying services it coordinates.
Roles	The facade exposes a small set of high-level operations such as analyzeImage() and analyzeProduct().
Why appropriate	Facade gives GUI/CLI controllers a simple entry point and hides workflow complexity.
What becomes harder without it	Without Facade, views/controllers would need to know the order and details of every subsystem call.

4.5 Observer

Design problem addressed	Long-running analysis stages should be able to report progress and results to the GUI without tightly coupling core services to GUI classes.
Participating classes	AnalysisSubject / AnalysisObserver (or equivalent event interface), ProductAnalysisFacade/Agent as publisher, NutriLensGUI as observer.
Roles	Observers receive events such as OCR complete, research complete, scoring complete, or analysis failed.
Why appropriate	Observer supports responsive UI updates and keeps analysis code independent of presentation.
What becomes harder without it	Without Observer, analysis services would need direct references to GUI elements or polling logic.

5. UML Use-Case Diagram

The use-case diagram shows the primary User actor and the relevant external-service actors. The major use cases collectively cover all fourteen features.



5.1 Suggested Major Use Cases

- UC01 — Analyze Product from Image
- UC02 — Analyze / Correct Product Manually
- UC03 — Review Nutrition and Score
- UC04 — Research an Ingredient
- UC05 — Compare Two Products
- UC06 — Ask Contextual Follow-Up Questions
- UC07 — Manage Saved Analyses
- UC08 — Get Automatic Alternatives
- UC09 — Calculate Macros for a Custom Serving

6. Detailed Use-Case Descriptions

UC01 — Analyze Product from Image

Actor(s)	User; OCR Service; AI/LLM Service; Research Service (when needed)
Goal	Analyze a packaged food from a label image and produce structured nutrition/ingredient information, a score, and an explanation.
Preconditions	Application is running; image input is available; required external services are configured.
Trigger	User uploads or captures a food-label image.
Main Success Scenario	1. User submits image. 2. System validates image. 3. OCR extracts text. 4. NutritionParser and IngredientParser structure the data. 5. User can confirm/correct extracted data. 6. NutritionAnalyzer evaluates the product. 7. ResearchAgent retrieves evidence when needed. 8. ScoringEngine calculates the score. 9. ExplanationAgent explains the breakdown. 10. Result is displayed and may be saved.
Alternative / Exception Flows	Unreadable image → request clearer image/manual entry. Missing values → mark unavailable. External research failure → continue with available deterministic analysis and disclose missing research.
Postconditions	A ProductAnalysis exists in current context; it may be saved, compared, questioned, or used to generate alternatives.
Related Feature(s)	F01, F02, F03, F04, F05, F06, F07, F08

UC02 — Analyze / Correct Product Manually

Actor(s)	User
Goal	Create or correct a product analysis without relying on successful OCR.
Preconditions	Application is running.
Trigger	User chooses manual entry or edits extracted fields.
Main Success Scenario	1. User enters/edits product fields. 2. InputValidator checks values. 3. Structured domain objects are created. 4. Nutrition/ingredient analysis runs. 5. ScoringEngine calculates the score. 6. Explanation is displayed.
Alternative / Exception Flows	Invalid or missing required values → identify exact fields and prevent finalization until resolved or explicitly marked unavailable.
Postconditions	Validated product data and ProductAnalysis are available.
Related Feature(s)	F02, F03, F04, F07, F08

UC03 — Review Nutrition and Score

Actor(s)	User
Goal	Understand a product's nutrition profile and why it received its score.
Preconditions	A product has been successfully analyzed.
Trigger	User opens the nutrition or score-breakdown view.

Main Success Scenario	1. System loads ProductAnalysis. 2. Nutrition metrics and score are displayed. 3. User requests explanation. 4. ExplanationAgent uses the deterministic ScoreBreakdown. 5. Explanation is displayed.
Alternative / Exception Flows	Explanation service unavailable → show deterministic factor list without generated prose.
Postconditions	User can continue to comparison, alternatives, research, or follow-up questions.
Related Feature(s)	F03, F07, F08

UC04 — Research an Ingredient

Actor(s)	User; Research Service; AI/LLM Service
Goal	Obtain an evidence-based summary for a selected ingredient.
Preconditions	Ingredient is known or selected from an analyzed product.
Trigger	User chooses Research or asks an ingredient-specific question.
Main Success Scenario	1. Agent identifies ingredient/question. 2. ResearchAgent plans retrieval. 3. ResearchTool retrieves sources. 4. EvidenceEvaluator filters/compares them. 5. Agent summarizes evidence and uncertainty. 6. Sources are shown with the response.
Alternative / Exception Flows	No reliable results → report insufficient evidence. Conflicting results → communicate disagreement. Tool failure → report retrieval failure.
Postconditions	IngredientEvidence is available in the current analysis/conversation.
Related Feature(s)	F05, F06, F10

UC05 — Compare Two Products

Actor(s)	User; AI/LLM Service (optional explanation)
Goal	Compare two analyzed products and understand their nutritional trade-offs.
Preconditions	At least two ProductAnalysis records exist.
Trigger	User selects two products and chooses Compare.
Main Success Scenario	1. System loads both analyses. 2. ComparisonService aligns comparable metrics. 3. Scores/nutrients/ingredients are shown side by side. 4. Optional AI explanation summarizes major differences.
Alternative / Exception Flows	Incompatible serving bases → normalize where possible and label assumptions. Missing data → show unavailable cells rather than inventing values.
Postconditions	Comparison remains available for follow-up questions or alternative selection.
Related Feature(s)	F09, F11, F12

UC06 — Ask Contextual Follow-Up Question

Actor(s)	User; AI/LLM Service; external tools as needed
Goal	Ask a natural-language question about the current product, score, ingredient, comparison, or recommendation.
Preconditions	Conversation context exists or user provides enough information.
Trigger	User sends a message in the chat/prompt interface.
Main Success Scenario	1. ConversationManager loads context. 2. AgentController interprets intent. 3. Existing analysis is used or a tool is invoked. 4. Agent produces a grounded response. 5. Conversation history is updated.
Alternative / Exception Flows	Ambiguous reference → ask clarification. Needed evidence unavailable → state limitation. Context missing → ask user to select a product.
Postconditions	Conversation history contains the new exchange.
Related Feature(s)	F10, F11

UC07 — Manage Saved Analyses

Actor(s)	User
Goal	Save, reopen, list, and reuse past product analyses.
Preconditions	Persistence is available.
Trigger	User chooses Save, History, Open, or Delete.
Main Success Scenario	1. Repository stores or retrieves ProductAnalysis. 2. History view refreshes. 3. User opens a record or selects it for comparison.
Alternative / Exception Flows	Storage error → inform user and keep current analysis in memory. Missing record → refresh history and report unavailable item.
Postconditions	Saved-product collection is updated or selected record is loaded.
Related Feature(s)	F12

UC08 — Get Automatic Alternatives

Actor(s)	User; Product Search Service; AI/LLM Service
Goal	Receive ranked comparable products with stronger nutritional characteristics.
Preconditions	Current product has a completed ProductAnalysis.
Trigger	Analysis completes automatically or user opens Alternatives.
Main Success Scenario	1. RecommendationService derives comparison criteria. 2. ProductSearchTool retrieves candidates. 3. Candidates are scored/analyzed. 4. RecommendationRanker ranks candidates. 5. ExplanationAgent describes key improvements/trade-offs. 6. Top alternatives are displayed.
Alternative / Exception Flows	No comparable candidates → report none found. Candidate data incomplete → exclude or label low-confidence candidate. User constraint conflict → filter and explain.
Postconditions	AlternativeRecommendation list is available for comparison or follow-up.
Related Feature(s)	F13, F09, F11

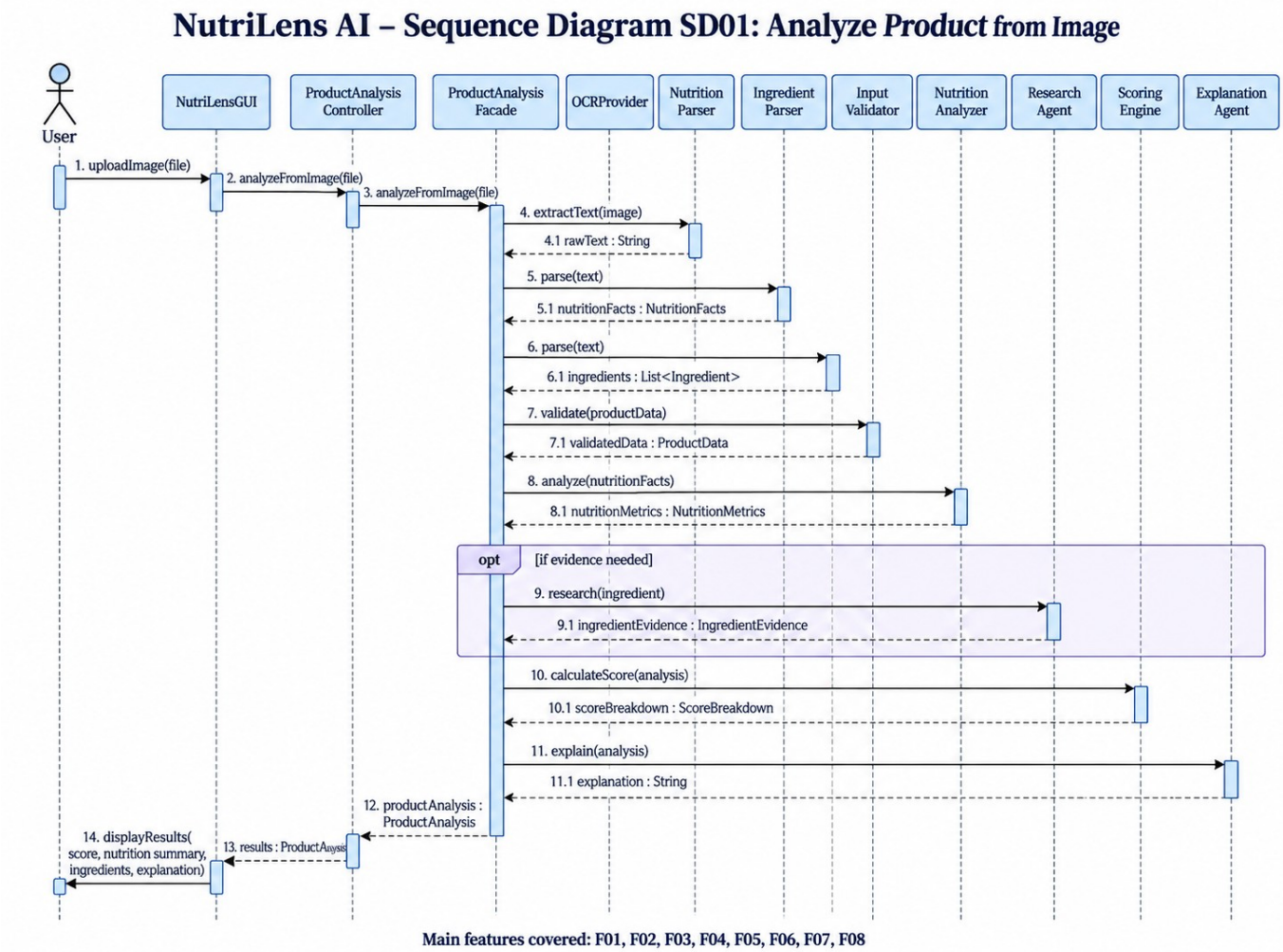
UC09 — Calculate Macros for a Custom Serving

Actor(s)	User
Goal	Calculate calories/macronutrients for an amount different from the package serving.
Preconditions	NutritionFacts includes a valid serving basis.
Trigger	User changes serving amount or number of servings.
Main Success Scenario	1. User enters custom amount. 2. ServingCalculator validates units/value. 3. Multiplier is calculated. 4. Scalable nutrients are recalculated. 5. Adjusted values are displayed.
Alternative / Exception Flows	Invalid amount → reject. Missing serving basis → explain that calculation cannot be completed. Unit mismatch → request compatible unit or conversion data.
Postconditions	Adjusted macro values are displayed without modifying the original label data.
Related Feature(s)	F14, F03

7. Sequence Diagrams

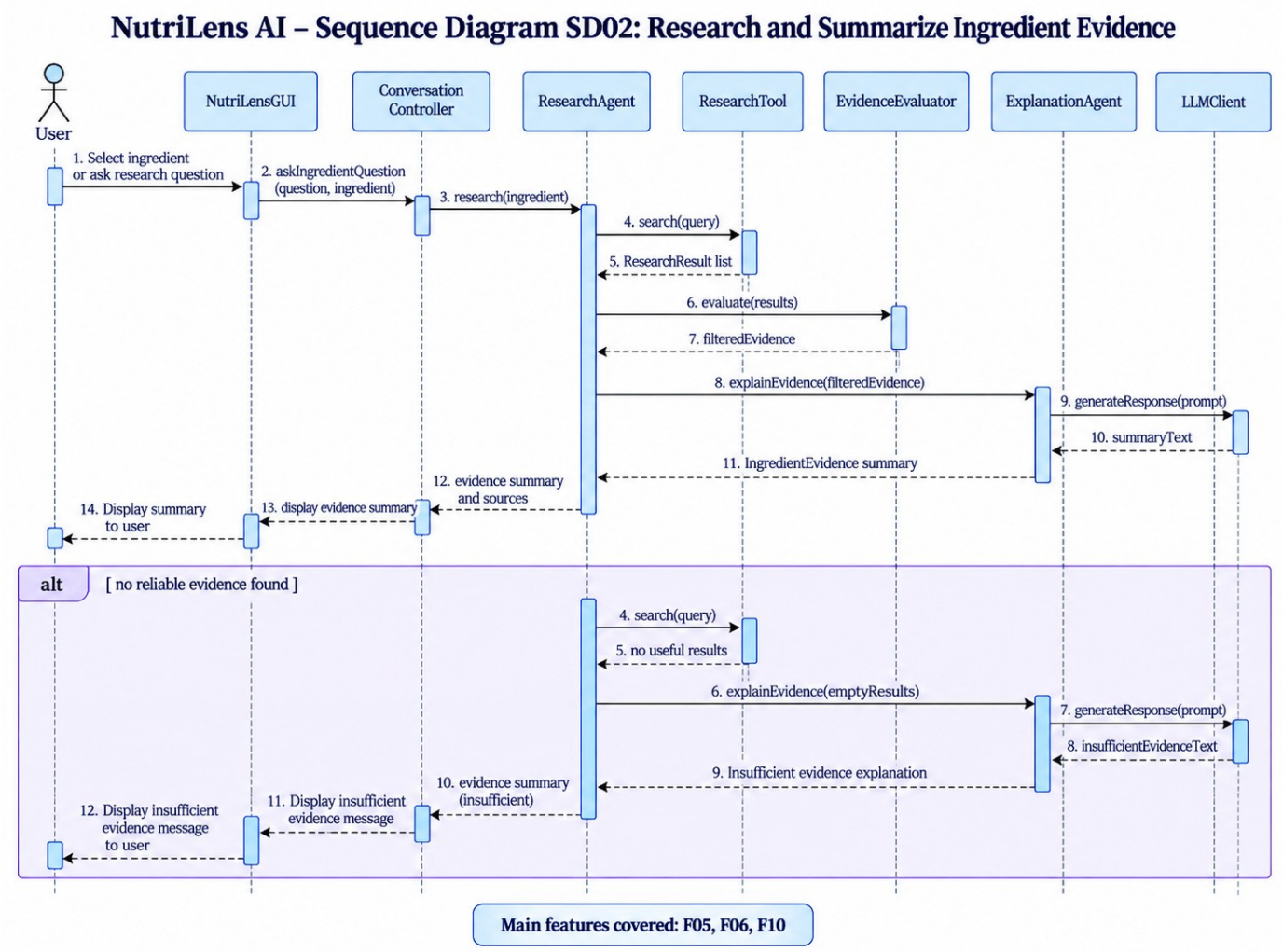
SD01 — Analyze Product from Image

User → NutriLensGUI/CLI → ProductAnalysisController → ProductAnalysisFacade → OCRProvider → NutritionParser / IngredientParser → InputValidator → NutritionAnalyzer → ResearchAgent (optional) → ScoringEngine → ExplanationAgent → UI.



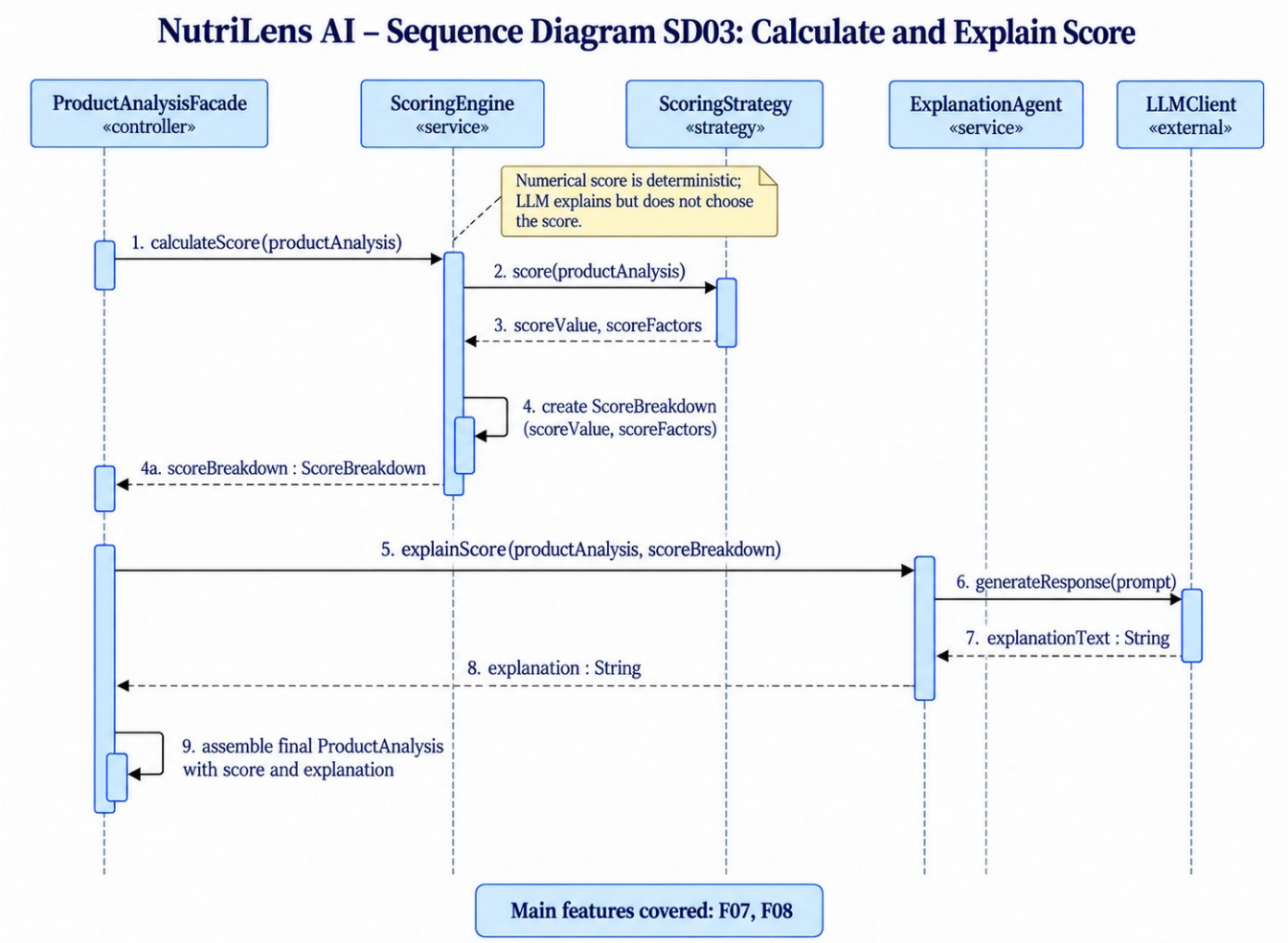
SD02 — Research and Summarize Ingredient Evidence

User/AnalysisFacade → ResearchAgent → ResearchTool → ResearchResult[] → EvidenceEvaluator → LLMClient → IngredientEvidence → User interface.



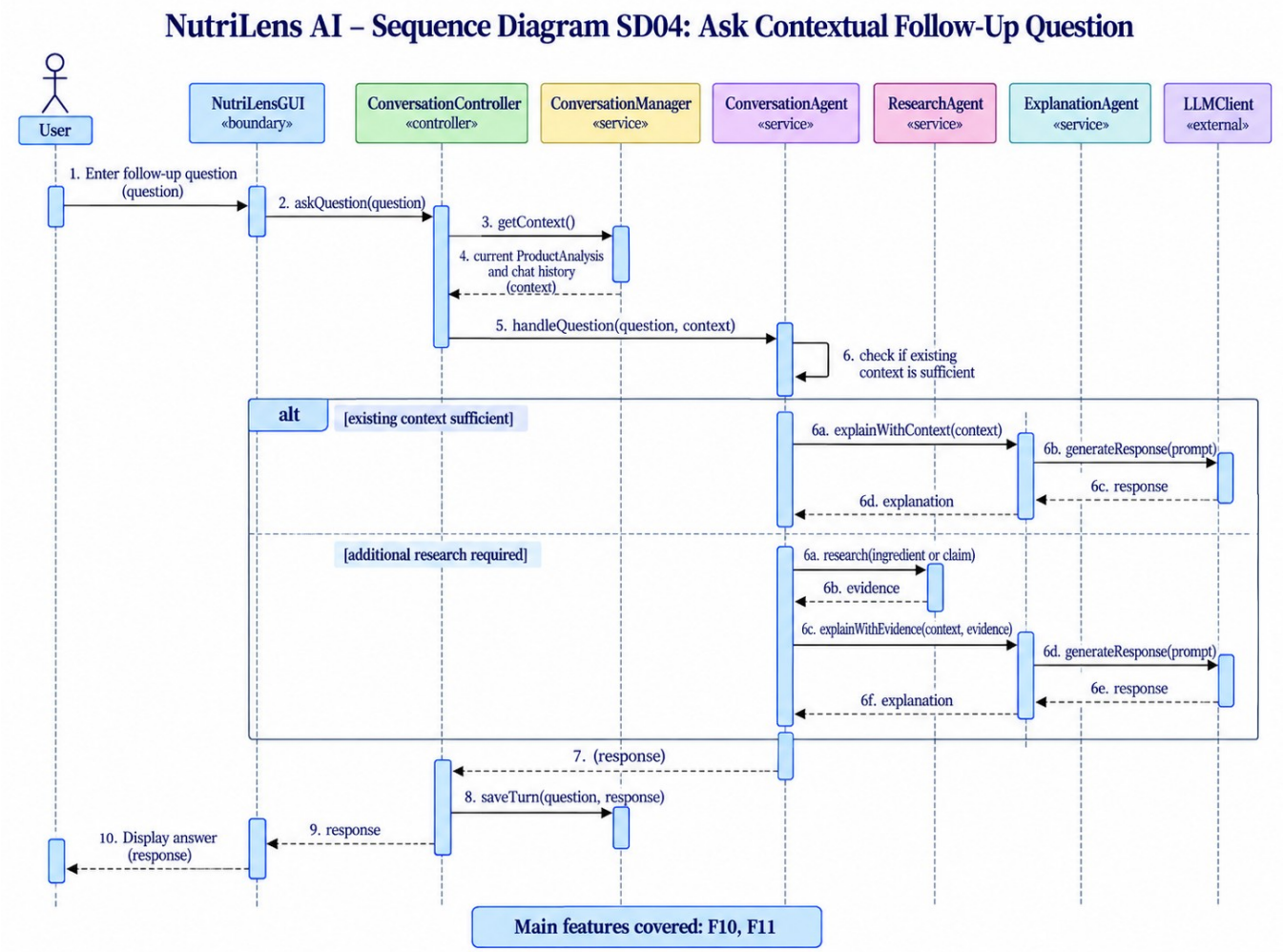
SD03 — Calculate and Explain Score

ProductAnalysisFacade → ScoringEngine → ScoringStrategy → ScoreBreakdown → ExplanationAgent → LLMClient → explanation → UI.



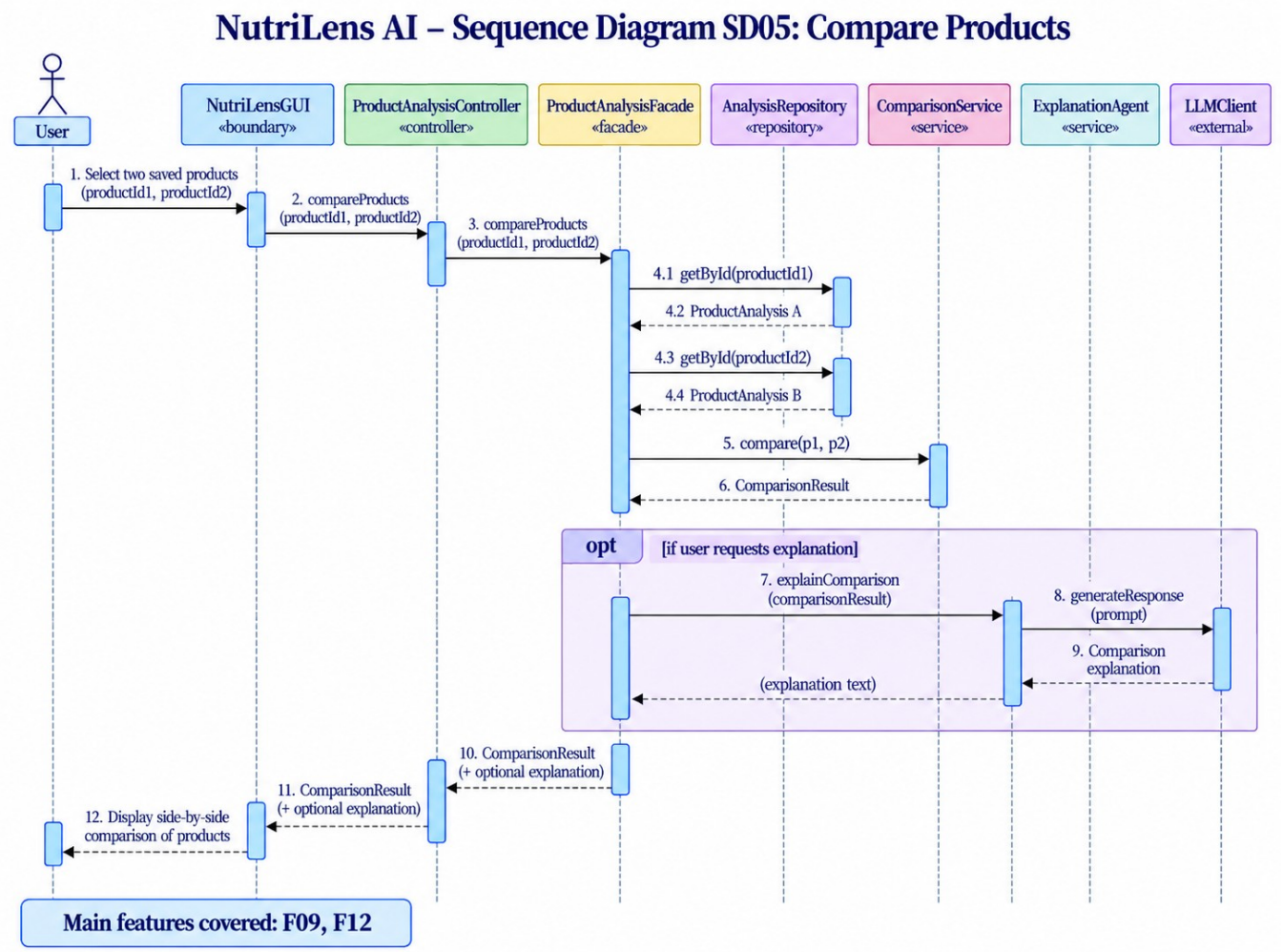
SD04 — Ask Contextual Follow-Up Question

User → ConversationController → ConversationManager → AgentController → analysis/research tool as needed → Explanation/ConversationAgent → LLMClient → response → ConversationManager → User.



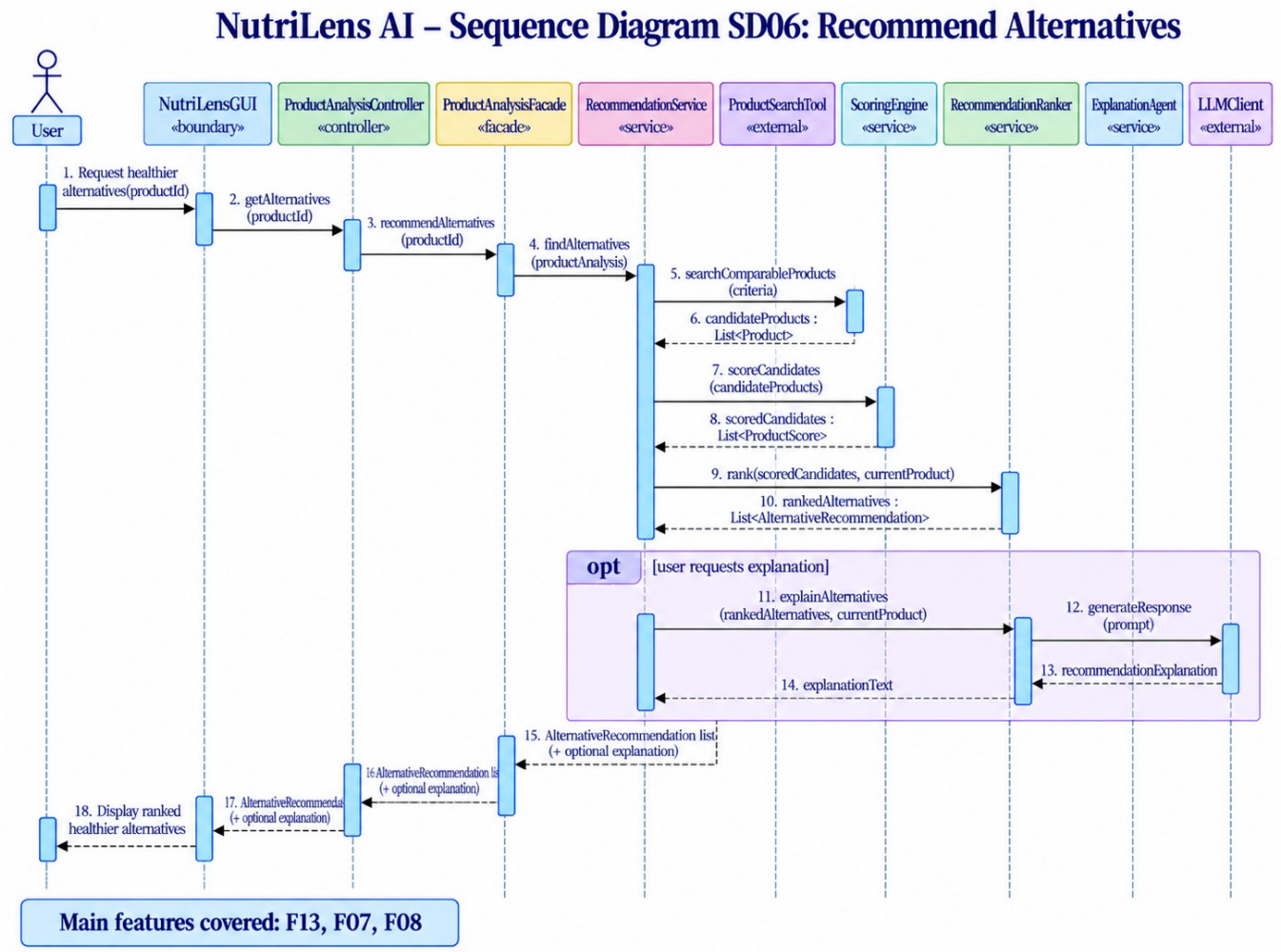
SD05 — Compare Products

User → Comparison view/controller → AnalysisRepository → ComparisonService → ProductAnalysis A/B → optional ExplanationAgent → comparison result → UI.



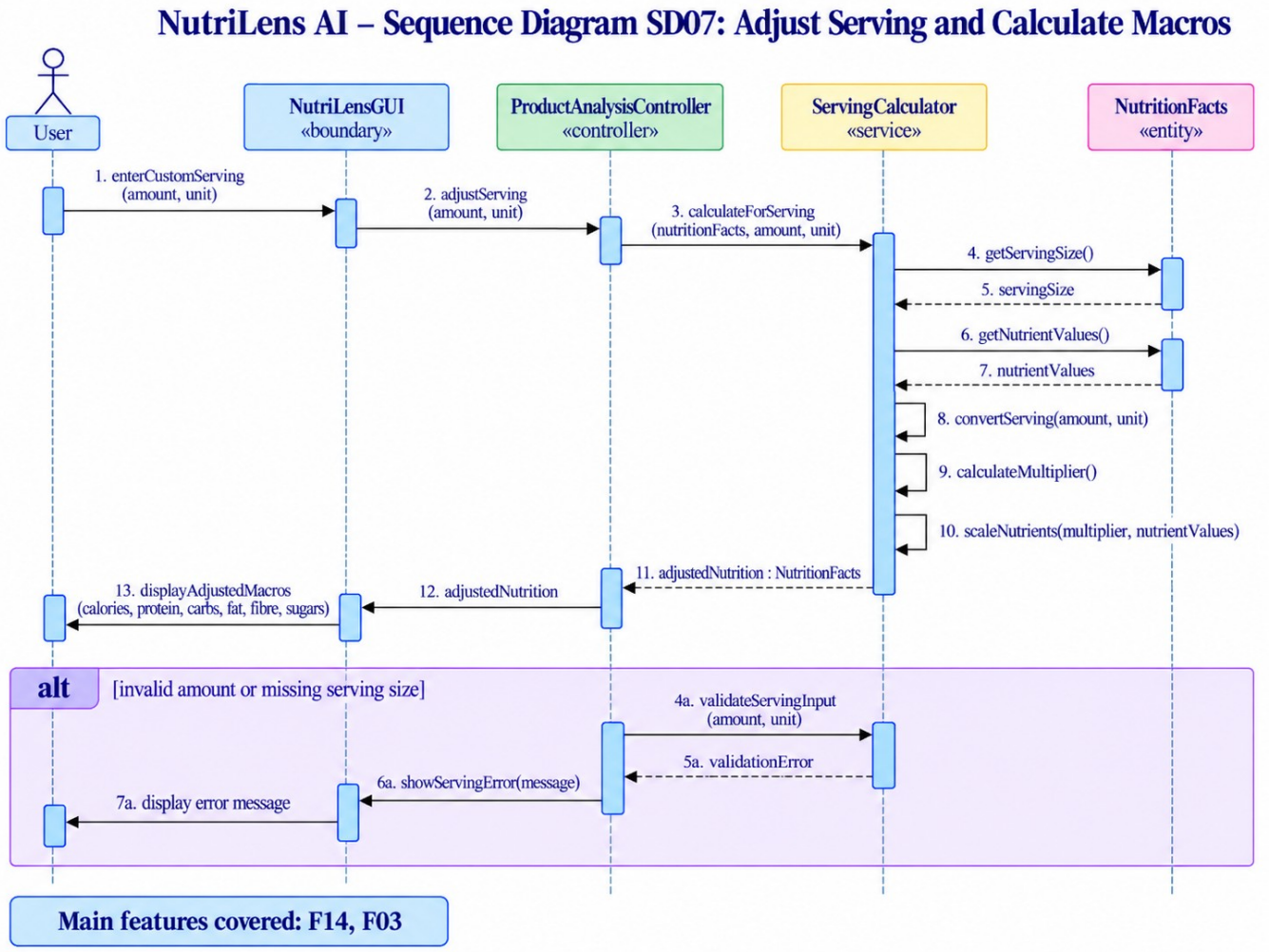
SD06 — Recommend Alternatives

ProductAnalysisFacade / User → RecommendationService → ProductSearchTool → candidate products → ScoringEngine → RecommendationRanker → ExplanationAgent → AlternativeRecommendation[] → UI.



SD07 — Adjust Serving and Calculate Macros

User → Nutrition view/controller → ServingCalculator → NutritionFacts → adjusted nutrient values → UI.


NutriLens AI – Sequence Diagram SD07: Adjust Serving and Calculate Macros
User
NutriLensGUI «boundary»
ProductAnalysisController «controller»
ServingCalculator «service»
NutritionFacts «entity»
1. enterCustomServing (amount, unit)
2. adjustServing (amount, unit)
3. calculateForServing (nutritionFacts, amount, unit)
4. getServingSize()
5. servingSize
6. getNutrientValues()
7. nutrientValues
8. convertServing(amount, unit)
9. calculateMultiplier()
10. scaleNutrients(multiplier, nutrientValues)
11. adjustedNutrition : NutritionFacts
12. adjustedNutrition
13. displayAdjustedMacros (calories, protein, carbs, fat, fibre, sugars)
alt
[invalid amount or missing serving size]
4a. validateServingInput (amount, unit)
5a. validationError
6a. showServingError(message)
7a. display error message
Main features covered: F14, F03

8. Feature-to-Design Traceability

This table maps each feature to its type, related use case, main classes, representative methods, sequence diagram, and design patterns. Update class/method names if the class diagram changes.

Feature	Description	Type	Use Case	Classes	Key Methods	Sequence	Pattern(s)
F01	Scan / Upload Food Label	Hybrid	UC01	NutriLensGUI/CLI; ProductAnalysisController; ProductAnalysisFacade; OCRProvider	uploadImage(); analyzeImage(); extractText()	SD01	MVC, Facade, Adapter
F02	Manual Entry & Correction	Deterministic	UC02	NutriLensGUI/CLI; InputValidator; ProductAnalysisController	submitProduct(); validate(); updateProduct()	SD01	MVC, Facade
F03	Nutrition Profile Analysis	Deterministic	UC01, UC03	NutritionAnalyzer; NutritionFacts; ProductAnalysis	analyzeNutrition(); normalize()	SD01, SD03	Facade
F04	Ingredient Analysis	Hybrid	UC01, UC04	IngredientParser; Ingredient; ResearchAgent	parseIngredients(); normalizeIngredient()	SD01, SD02	Facade, Adapter
F05	Scientific Evidence Retrieval	AI/Hybrid	UC04	ResearchAgent; ResearchTool	researchIngredient(); search()	SD02	Adapter, Facade
F06	Evidence Evaluation & Summary	AI	UC04	EvidenceEvaluator; IngredientEvidence; LLMClient	evaluate(); summarize(); generate()	SD02	Adapter
F07	1–100 Nutritional Score	Deterministic	UC01, UC03	ScoringEngine; ScoringStrategy; ScoreBreakdown	calculateScore(); score()	SD03	Strategy
F08	Explainable Score Breakdown	Hybrid	UC03	ExplanationAgent; ScoreBreakdown; LLMClient	explainScore(); generate()	SD03	Adapter, Facade
F09	Product Comparison	Hybrid	UC05	ComparisonService; ProductAnalysis; ExplanationAgent	compare(); explainTradeoffs()	SD05	Facade
F10	Ingredient Question Answering	AI	UC04, UC06	ConversationController; ResearchAgent; ConversationAgent	askIngredient(); answer(); search()	SD02, SD04	MVC, Adapter
F11	Contextual Follow-Up Conversation	AI	UC06	ConversationManager; AgentController; ConversationAgent	getContext(); handleMessage(); saveTurn()	SD04	MVC, Observer
F12	Analysis History / Saved Products	Deterministic	UC07	AnalysisRepository; ProductAnalysis; NutriLensGUI/CLI	save(); load(); list(); delete()	SD05	MVC
F13	Automatic Alternatives	Hybrid	UC08	RecommendationService; ProductSearchTool; RecommendationRanker; ScoringEngine	findAlternatives(); search(); rank()	SD06	Adapter, Strategy, Facade
F14	Macro Calculator & Serving Adjustment	Deterministic	UC09	ServingCalculator; NutritionFacts	calculateForServing(); convertServing()	SD07	MVC

9. Feature Implementation Explanations

Each subsection below connects a feature to its use case, sequence diagram, participating classes, representative methods, and runtime collaboration. These are design-level explanations and should be updated if the UML design changes.

F01 — Scan / Upload Food Label

Related Use Case	UC01
Related Sequence Diagram	SD01
Classes Involved	NutriLensGUI/CLI; ProductAnalysisController; ProductAnalysisFacade; OCRProvider; NutritionParser; IngredientParser
Important Methods	uploadImage(); analyzeImage(); extractText(); parseNutrition(); parseIngredients()

Execution: The user provides an image through the GUI or CLI. The controller delegates the analysis request to ProductAnalysisFacade, which invokes the OCR adapter and parsers. Extracted fields are returned for validation/correction before later analysis stages continue.

F02 — Manual Product Entry & OCR Correction

Related Use Case	UC02
Related Sequence Diagram	SD01
Classes Involved	NutriLensGUI/CLI; ProductAnalysisController; InputValidator; Product
Important Methods	submitProduct(); validate(); updateProduct()

Execution: Manual or corrected fields are submitted to the controller. InputValidator checks ranges, units, and required values before the controller creates or updates the Product and NutritionFacts objects.

F03 — Nutrition Profile Analysis

Related Use Case	UC01, UC03
Related Sequence Diagram	SD01 / SD03
Classes Involved	NutritionAnalyzer; NutritionFacts; ProductAnalysis
Important Methods	analyzeNutrition(); normalize()

Execution: After validation, NutritionAnalyzer derives normalized nutrition metrics from NutritionFacts and stores them in ProductAnalysis. The GUI and scoring engine reuse the same structured analysis.

F04 — Ingredient Analysis

Related Use Case	UC01, UC04
Related Sequence Diagram	SD01 / SD02
Classes Involved	IngredientParser; Ingredient; ResearchAgent
Important Methods	parseIngredients(); normalizeIngredient()

Execution: IngredientParser separates and normalizes ingredient entries. The ordered Ingredient collection is stored with the Product and can be passed to ResearchAgent when evidence or ingredient-specific questions are requested.

F05 — Scientific Evidence Retrieval

Related Use Case	UC04
Related Sequence Diagram	SD02
Classes Involved	ResearchAgent; ResearchTool; ResearchResult

Important Methods	researchIngredient(); search()

Execution: ResearchAgent determines what evidence is needed and invokes the ResearchTool interface. A provider-specific adapter retrieves results and converts them into ResearchResult objects used by later evaluation.

F06 — Evidence Evaluation & Summary

Related Use Case	UC04
Related Sequence Diagram	SD02
Classes Involved	EvidenceEvaluator; IngredientEvidence; LLMClient
Important Methods	evaluate(); summarize(); generate()

Execution: EvidenceEvaluator filters retrieved information and prepares grounded context. The LLM summarizes relevant findings and uncertainty into IngredientEvidence while deterministic checks ensure required source metadata is present.

F07 — 1–100 Nutritional Score

Related Use Case	UC01, UC03
Related Sequence Diagram	SD03
Classes Involved	ScoringEngine; ScoringStrategy; ScoreBreakdown
Important Methods	calculateScore(); score()

Execution: ScoringEngine delegates the calculation to the active ScoringStrategy. The strategy returns component contributions and a bounded 1–100 score. The numerical result is deterministic and does not depend on the LLM.

F08 — Explainable Score Breakdown

Related Use Case	UC03
Related Sequence Diagram	SD03
Classes Involved	ExplanationAgent; ScoreBreakdown; LLMClient
Important Methods	explainScore(); generate()

Execution: ExplanationAgent receives the completed ScoreBreakdown and nutrition/evidence context. It asks the LLM to explain only verified score factors. If generation fails, the system can display the deterministic factor list directly.

F09 — Product Comparison

Related Use Case	UC05
Related Sequence Diagram	SD05
Classes Involved	ComparisonService; ProductAnalysis; ExplanationAgent
Important Methods	compare(); explainTradeoffs()

Execution: ComparisonService loads two analyses and aligns comparable metrics. Numeric differences are calculated deterministically, while ExplanationAgent can summarize the most important trade-offs in plain language.

F10 — Ingredient Question Answering

Related Use Case	UC04, UC06
Related Sequence Diagram	SD02 / SD04
Classes Involved	ConversationController; ResearchAgent; ConversationAgent
Important Methods	askIngredient(); handleMessage(); answer()

Execution: The user asks a question in the ingredient or conversation interface. The agent resolves the ingredient context, uses stored evidence when sufficient, and invokes new research when required before generating a grounded response.

F11 — Contextual Follow-Up Conversation

Related Use Case	UC06
Related Sequence Diagram	SD04
Classes Involved	ConversationManager; AgentController; ConversationAgent
Important Methods	getContext(); handleMessage(); saveTurn()

Execution: ConversationManager supplies the active product, analysis, comparison, and recent exchanges. AgentController routes the request to the correct analysis or research path and stores the completed response back into conversation history.

F12 — Analysis History / Saved Products

Related Use Case	UC07
Related Sequence Diagram	SD05
Classes Involved	AnalysisRepository; ProductAnalysis; NutriLensGUI/CLI
Important Methods	save(); load(); list(); delete()

Execution: The repository persists completed ProductAnalysis records. GUI and CLI views can list, reopen, remove, or select saved products for comparison without duplicating storage logic.

F13 — Automatic Alternative Recommendations

Related Use Case	UC08
Related Sequence Diagram	SD06
Classes Involved	RecommendationService; ProductSearchTool; RecommendationRanker; ScoringEngine; ExplanationAgent
Important Methods	findAlternatives(); search(); rank(); explainAlternative()

Execution: RecommendationService derives criteria from the current product, retrieves comparable candidates through ProductSearchTool, reuses ScoringEngine to evaluate candidates, and ranks them. ExplanationAgent describes the differences without inventing products.

F14 — Macro Calculator & Serving Adjustment

Related Use Case	UC09
Related Sequence Diagram	SD07
Classes Involved	ServingCalculator; NutritionFacts; NutriLensGUI/CLI
Important Methods	calculateForServing(); convertServing()

Execution: The user provides a custom amount. ServingCalculator computes a multiplier from the label serving basis and scales applicable nutrient values. The original label values remain unchanged.

Appendix A. Proposed Class Inventory

Class / Interface	Primary Responsibility
NutriLensGUI	GUI view for major functionality.
NutriLensCLI	CLI view exposing the same major functionality.
ProductAnalysisController	Coordinates analysis-related user requests.
ConversationController	Coordinates conversational requests.
ProductAnalysisFacade	High-level facade for the complete product-analysis workflow.
Product	Represents a food product.
NutritionFacts	Stores serving and nutrient values.
Ingredient	Represents one ingredient and its normalized name.
ProductAnalysis	Aggregates analysis results for a product.
ScoreBreakdown	Stores total score and component contributions.
IngredientEvidence	Stores evaluated research evidence and uncertainty.
AlternativeRecommendation	Represents a recommended comparable product and explanation.
InputValidator	Validates manual/OCR-derived structured input.
OCRProvider	Interface for OCR adapters.
NutritionParser	Parses nutrition-label text.
IngredientParser	Parses ingredient-list text.
NutritionAnalyzer	Produces normalized nutrition analysis.
ServingCalculator	Calculates values for custom serving sizes.
ScoringStrategy	Strategy interface for scoring algorithms.
StandardScoringStrategy	Initial scoring implementation.
ScoringEngine	Coordinates deterministic score calculation.
ResearchTool	Interface for research/search adapters.
ResearchAgent	Plans and executes ingredient research.
EvidenceEvaluator	Evaluates retrieved evidence for relevance/consistency.
LLMClient	Interface for the selected LLM provider.
ExplanationAgent	Generates grounded explanations.
ConversationAgent	Handles contextual conversational reasoning.
ConversationManager	Maintains current conversation/product context.
ProductSearchTool	Interface for retrieving alternative products.
RecommendationService	Coordinates alternative recommendation workflow.
RecommendationRanker	Ranks candidate alternatives.
ComparisonService	Compares two ProductAnalysis records.
AnalysisRepository	Persists and retrieves saved analyses.

Appendix B. Finalized Scoring Model and Planned Technology Choices

This appendix records the finalized Stage 1 scoring logic and the planned external tools that will be used during Stage 2 implementation.

B1. Finalized 1–100 Scoring Model

The NutriLens AI numerical score is deterministic. The LLM explains the score, but it does not generate the score itself. The score begins with a neutral base value and is adjusted using defined rules that can be implemented and tested directly in the ScoringEngine.

- Base score: 50, providing a neutral starting point.
- Positive nutrition factors: up to +25 total, based on fibre (up to +10), protein (up to +8), and selected micronutrients (up to +7).

- Negative nutrition factors: up to −30 total, based on added sugar (up to −12), sodium (up to −10), and saturated fat (up to −8).
- Ingredient quality adjustment: up to −15 total for highly processed ingredients, artificial additives or low-quality fillers, and excessive ingredient complexity.
- Score bounds: the final score is capped between 1 and 100.
- Missing values are marked unavailable and are not invented by the system.
- Role of scientific evidence: research evidence mainly supports explanation and ingredient discussion; it does not directly invent or dominate the numerical score. Conflicting evidence is communicated as uncertainty.

Scoring Formula: Final Score = Base Score + Positive Factors − Negative Factors − Ingredient Penalties

B2. Planned External Tools and Technology Choices

The following technologies are the planned integrations for Stage 2 implementation.

Component	Planned Tool / Technology
GUI Framework	JavaFX
CLI	Java Console Menu Interface
LLM Provider	OpenAI API (via LLMClient adapter)
OCR Provider	Tesseract OCR (via OCRProvider adapter)
Research Source	PubMed / Europe PMC search
Product Data Source	Open Food Facts API
Persistence	SQLite or JSON-based repository
Architecture Note	External services are accessed through adapters so providers can be replaced without changing core business logic.

B3. Design Rationale

- Deterministic scoring improves testability, transparency, and consistency.
- Adapter-based integrations reduce coupling to external APIs and support future provider replacement.
- AI is used for reasoning, explanation, retrieval planning, and conversation, rather than uncontrolled numeric scoring.